

Inheritance: Extending Classes

8

Key Concepts

Reusability | Inheritance | Single inheritance | Multiple inheritance | Multilevel inheritance | Hybrid inheritance | Hierarchical inheritance | Defining a derived class | Inheriting private members | Virtual base class | Direct base class | Indirect base class | Abstract class | Defining derived class constructors | Nesting of classes

INTRODUCTION

Reusability is an important feature of object oriented programming (OOP). Reusing an existing piece of code offers manifold benefits. It not only saves the effort and cost required to build a software product but also enhances its reliability. Therefore, programmers need not re-write, re-debug, and re-test the code that has already been tested and being used in existing software.

To support reusability, C++ supports the concept of reusing existing classes as it allows programmers to create new classes that reuse prewritten and tested classes. The existing classes are adapted as per user's requirements so that the newly formed classes can be incorporated in the current software application being developed.

(The technique of creating a new class from an existing class is called inheritance.) The old or existing class is called the base class and the new class is known as the derived class or sub-class. (The derived classes are created by first inheriting the data and methods of the base class and then adding new specialized data and functions in it.) During the process of inheritance, the base class remains unchanged. The concept of inheritance is, therefore, frequently used to implement the is-a relationship.

For example, teacher is-a person, student is-a person; while both teacher and student are a person in the first place, both also have some distinguishing features. Therefore, all the common traits of teacher and student are specified in the Person class and specialized features are incorporated in two separate classes of Teacher and Student.

Similarly, a dentist or a surgeon is a doctor and doctor is a person. Fig. 12.1 illustrates the concept of inheritance which follows a top-down approach to problem solving. In top-down design approach, generalized classes are designed first and specialized classes are derived by inheriting or extending generalized classes.

12.1 DEFINING DERIVED CLASSES

A class can be derived from one or more base classes using the following syntax:

```
class derived_class_name : [access-specifier] base_class_name
{
    -----
    -----
    -----
};
```

According to the syntax, the `derived_class_name` is derived from the `base_class_name`, thereby inheriting some or all of its members. The access-specifier, also known as the visibility-mode, is optional and if present, can be either `public`, `private`, or `protected`. If no access-specifier is written, then by default, the class will be derived in private mode.

✓ **Table 12.1** Visibility modes

Specifier or class	Same class	Derived class	Any other class	Friend function	Friend class
Private	Yes	No	No	Yes	Yes
Protected	Yes	Yes	No	Yes	Yes
Public	Yes	Yes	Yes	Yes	Yes

Note The keywords `private`, `protected`, and `public` may appear in any order and any number of times in class declaration.

12.2.1 Inheriting Protected Members

When a protected member is inherited in private mode, the protected member of the base class becomes private member of the derived class. Although these members can be accessed in the derived class, these are not available for further inheritance (in case of multi-level inheritance) because private members cannot be inherited as shown in Fig. 12.2.

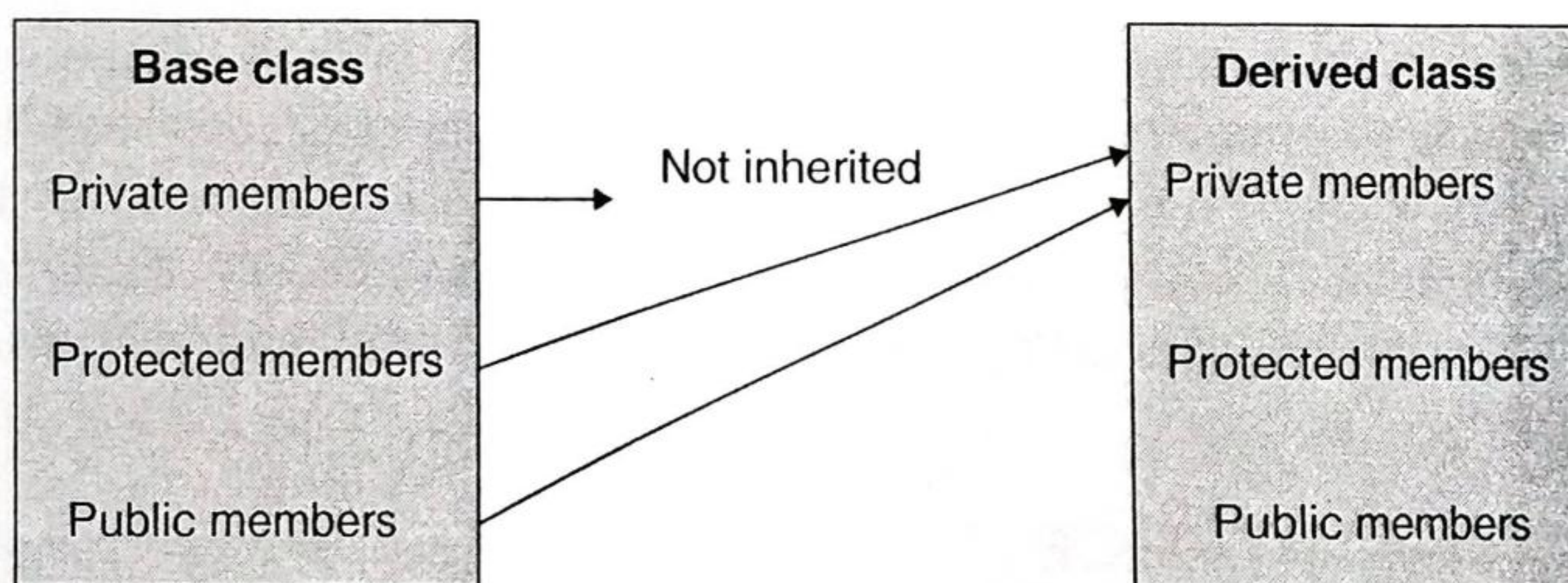


Figure 12.2 A class derived in private mode

However, when a protected member is inherited in public mode, it becomes a protected member of the derived class. Therefore, it can be easily accessed in the derived class and can be used for further inheritance as shown in Fig. 12.3.

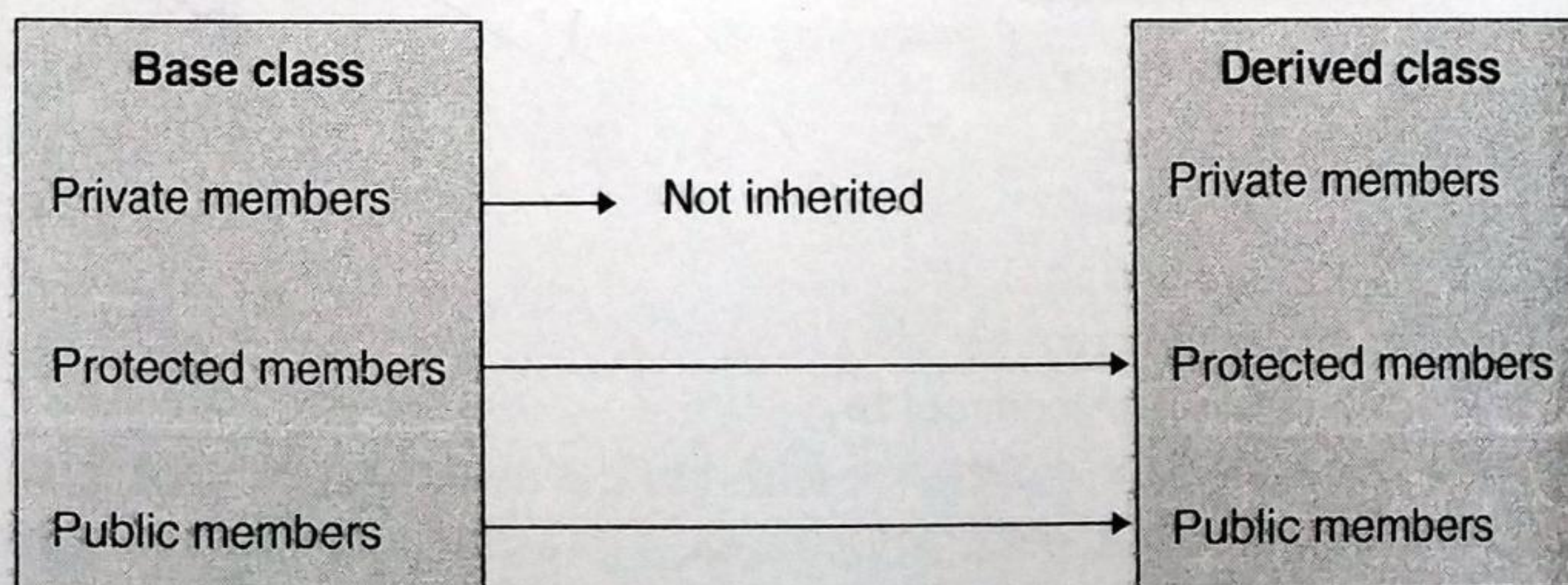


Figure 12.3 A class derived in public mode

12.2.2 Inheriting the Class in Protected Mode

A base class can be inherited in protected mode. This type of derivation is known as protected derivation. When a class is inherited in protected mode, the public and protected members of the class becomes protected members of the derived class as shown in Fig. 12.4.

Programming Tip: Avoid the protected label whenever you can attain the same result with the private label.

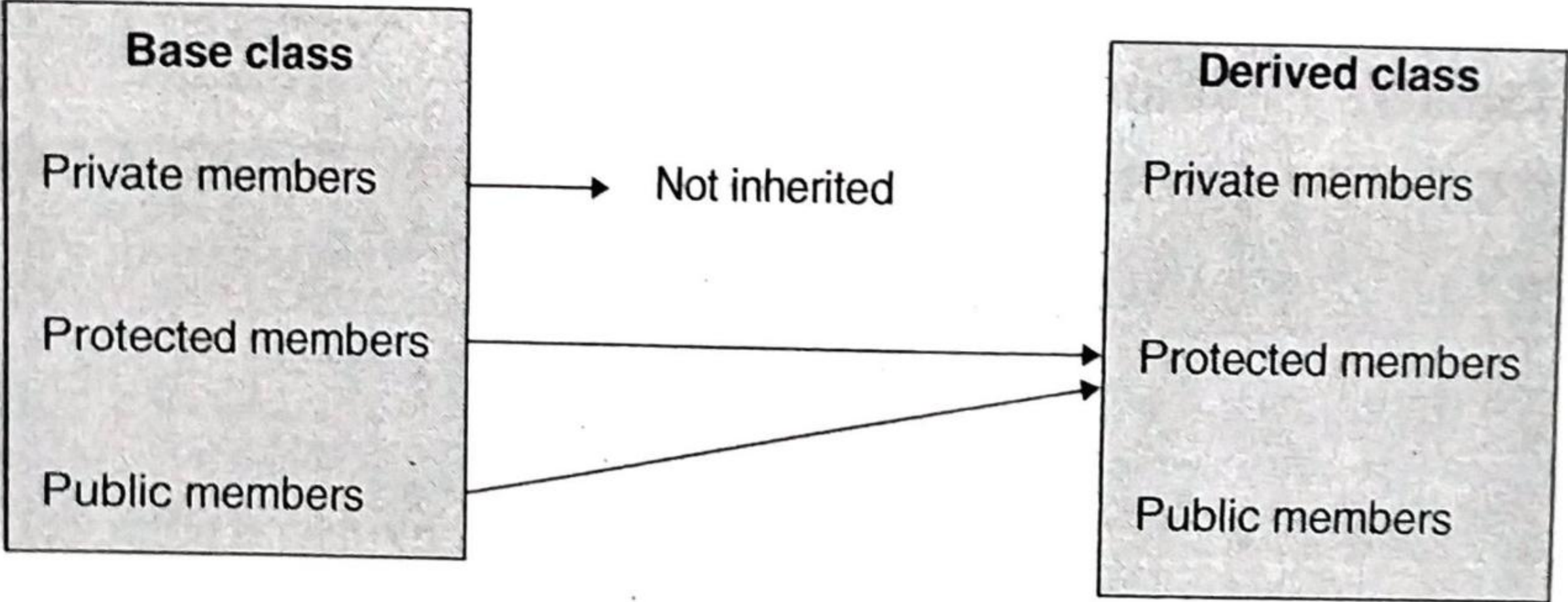
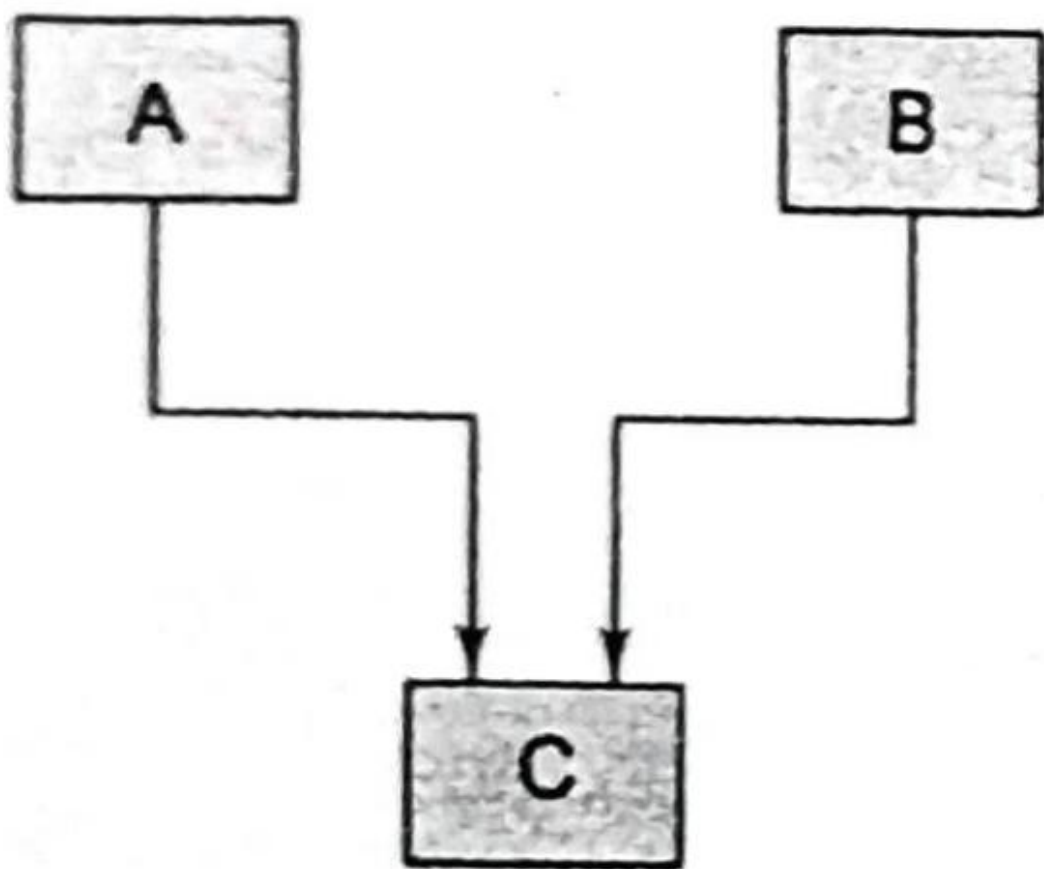


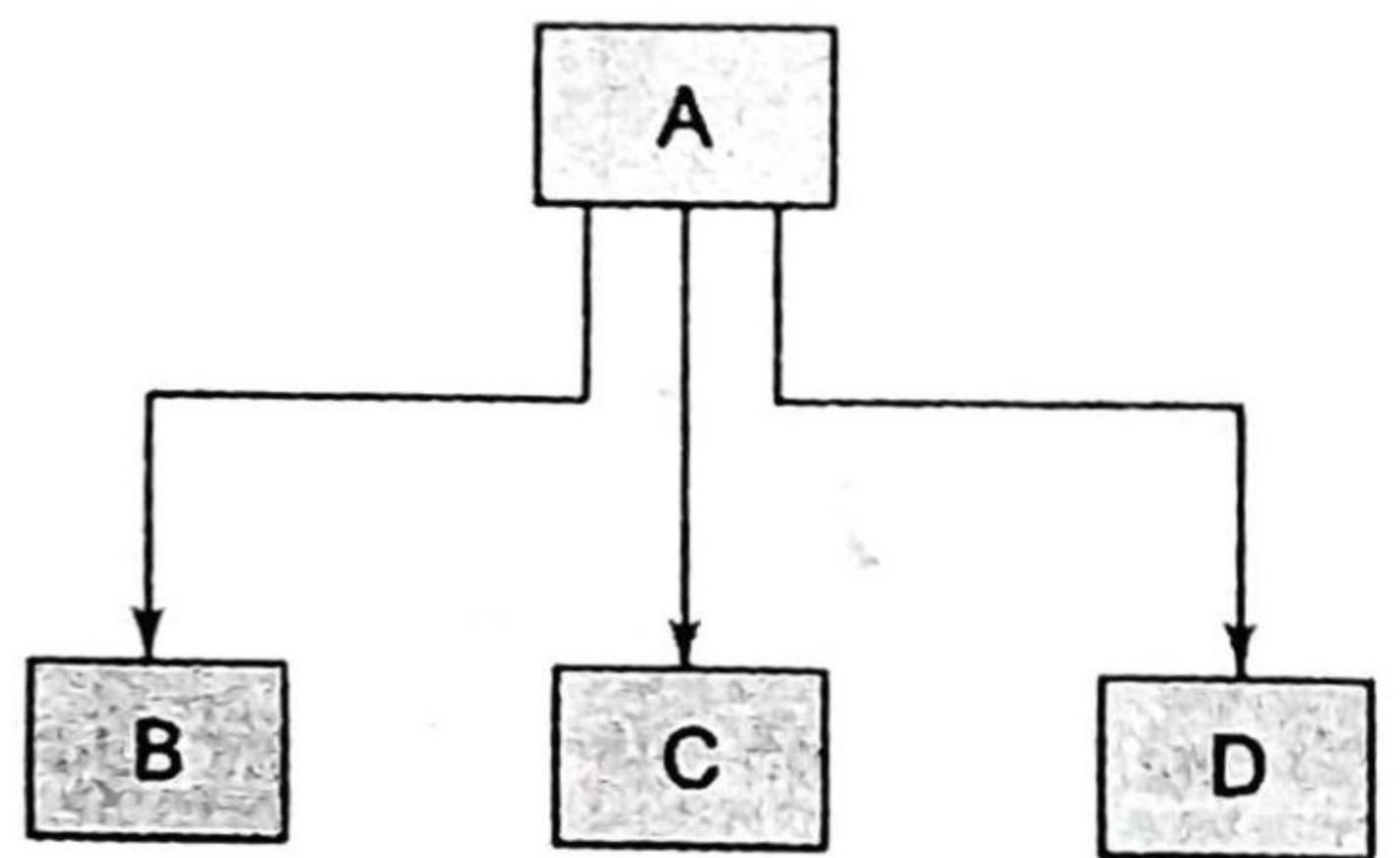
Figure 12.4 A class derived in protected mode



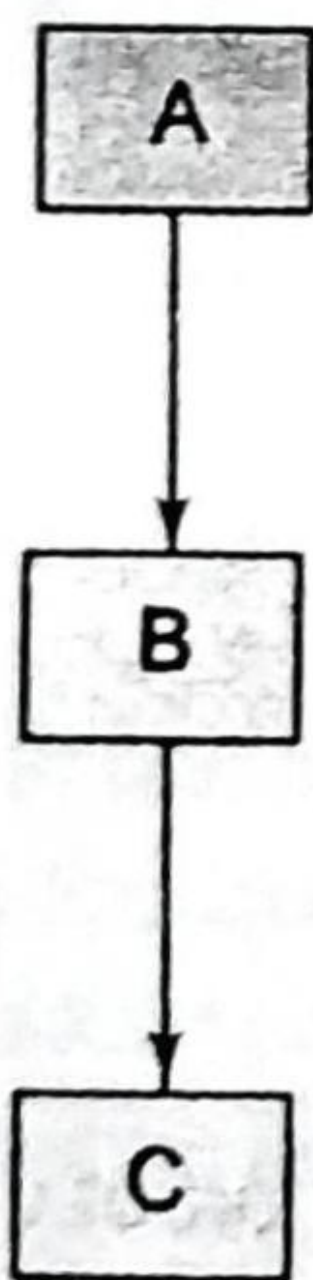
(a) Single inheritance



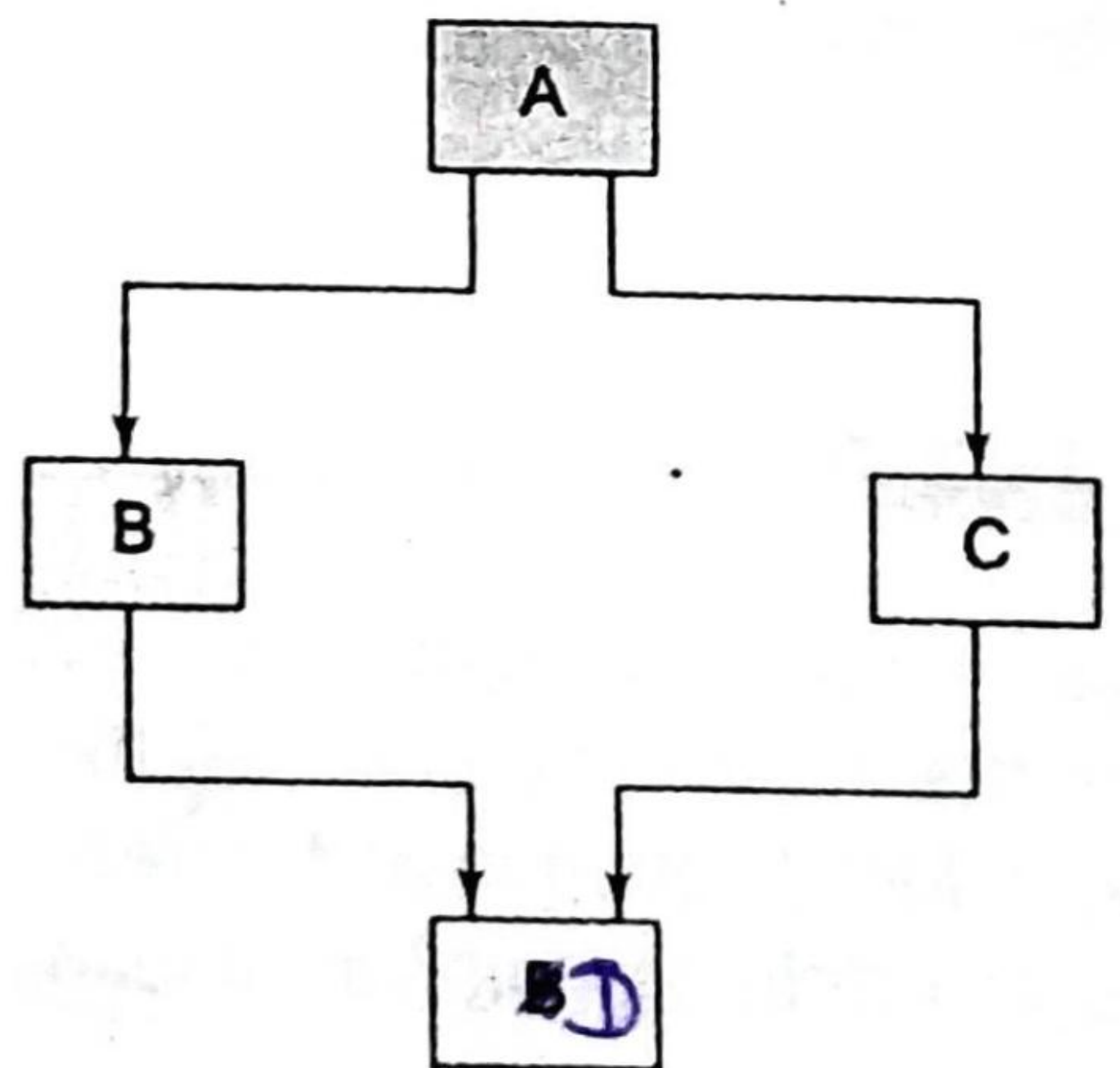
(b) Multiple inheritance



(c) Hierarchical inheritance



(d) Multilevel inheritance



(e) Hybrid inheritance

Fig. 8.1 Forms of inheritance



8.2 DEFINING DERIVED CLASSES

A derived class can be defined by specifying its relationship with the base class in addition to its own details. The general form of defining a derived class is:

```
class derived-class-name : visibility-mode base-class-name
{
    ..... //
    ..... // members of derived class
    ..... //
};
```

The colon indicates that the *derived-class-name* is derived from the *base-class-name*. The *visibility-mode* is optional and, if present, may be either **private** or **public**. The default visibility-mode is **private**. Visibility mode specifies whether the features of the base class are *privately derived* or *publicly derived*.


```

class derived: private base    // private derivation
{
    members of derived
};
class derived: public base    // public derivation
{
    members of derived
};
class derived: base           // private derivation by default
{
    members of derived
};

```

When a base class is privately inherited by a derived class, 'public members' of the base class become 'private members' of the derived class and therefore the public members of the base class can only be accessed by the member functions of the derived class. They are inaccessible to the objects of the derived class. Remember, a public member of a class can be accessed by its own objects using the dot operator. The result is that no member of the base class is accessible to the objects of the derived class.

On the other hand, when the base class is *publicly inherited*, 'public members' of the base class become 'public members' of the derived class and therefore they are accessible to the objects of the derived class. In both the cases, the *private members* are not inherited and therefore, the private members of a base class will never become the members of its derived class.



8.3 SINGLE INHERITANCE

Let us consider a simple example to illustrate inheritance. Program 8.1 shows a base class **B** and a derived class **D**. The class **B** contains one private data member, one public data member, and three public member functions. The class **D** contains one private data member and two public member functions.

Program 8.1

Single Inheritance: Public

```
#include <iostream>
using namespace std;

class B
{
    int a;    // private; not inheritable
public;
```



```

        int b;    // public; ready for inheritance
        void set_ab();
        int get_a(void);
        void show_a(void);
    };

    class D : public B    // public derivation
    {
        int c;
    public:
        void mul(void);
        void display(void);
    };

    //-----
    void B :: set_ab(void)
    {
        a = 5; b = 10;
    }

    int B :: get_a()
    {
        return a;
    }

    void B :: show_a()
    {
        cout << "a = " << a << "\n";
    }

    void D :: mul()
    {
        c = b * get_a();
    }

    void D :: display()
    {
        cout << "a = " << get_a() << "\n";
        cout << "b = " << b << "\n";
        cout << "c = " << c << "\n\n";
    }

    //-----
    int main()
    {
        D d;

        d.set_ab();
        d.mul();
        d.show_a();
        d.display();

        d.b = 20;
        d.mul();
        d.display();

        return 0;
    }

```


The output of Program 8.1 would be:

```
a = 5  
a = 5  
b = 10  
c = 50
```

```
a = 5  
b = 20  
c = 100
```

The class **D** is a public derivation of the base class **B**. Therefore, **D** inherits all the **public** members of **B** and retains their visibility. Thus, a **public** member of the base class **B** is also a public member of the derived class **D**. The private members of **B** cannot be inherited by **D**. The class **D**, in effect, will have more members than what it contains at the time of declaration as shown in Fig. 8.2.

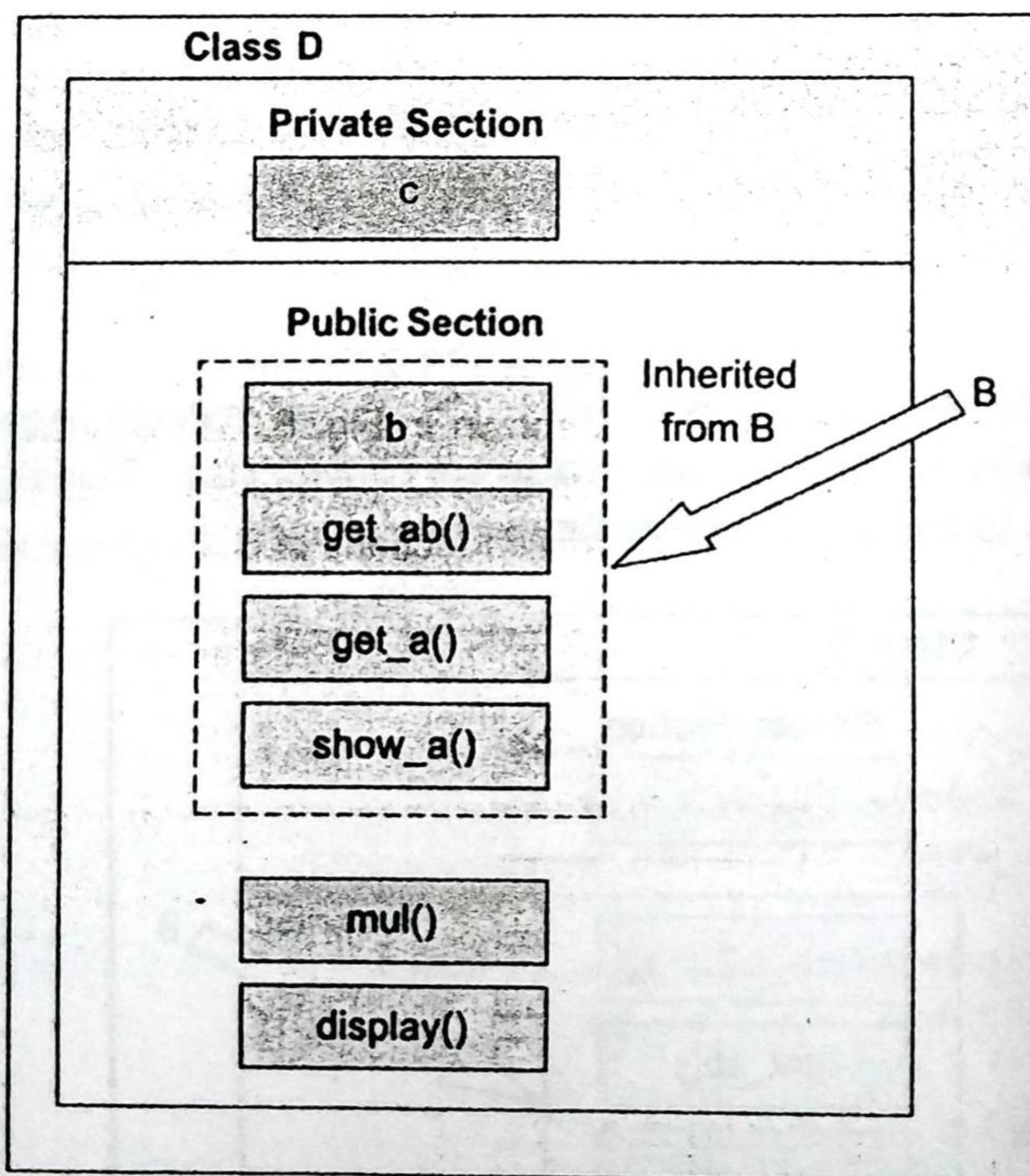


Fig. 8.2 Adding more members to a class (by public derivation)

Program 8.2

Single Inheritance : Private

```
#include <iostream>

using namespace std;

class B
{
    int a;                // private; not inheritable
public:
    int b;                // public; ready for inheritance
    void get_ab();
    int get_a(void);
    void show_a(void);
};

class D : private B       // private derivation
{
    int c;
public:
    void mul(void);
    void display(void);
};
```



```

// -----
void B :: get_ab(void)
{
    cout << "Enter values for a and b: ";
    cin >> a >> b;
}

int B :: get_a()
{
    return a;
}

void B :: show_a()
{
    cout << "a = " << a << "\n";
}

void D :: mul()
{
    get_ab();
    c = b * get_a();           // 'a' cannot be used directly
}

void D :: display()
{
    show_a();                  // outputs value of 'a'
    cout << "b = " << b << "\n"
        << "c = " << c << "\n\n";
}

// -----
int main()
{
    D d;

    // d.get_ab(); WON'T WORK
    d.mul();
    // d.show_a(); WON'T WORK
    d.display();
    // d.b = 20; WON'T WORK; b has become private
    d.mul();
    d.display();

    return 0;
}

```

The output of Program 8.2 would be:

```

Enter values for a and b: 5 10
a = 5

```



```
b = 10
c = 50
Enter values for a and b: 12 20
a = 12
b = 20
c = 240
```

Suppose a base class and a derived class define a function of the same name. What will happen when a derived class object invokes the function? In such cases, the derived class function supersedes the base class definition. However, if the derived class does not redefine the function, then the base class function will be called.